

MATERIAL DE APOIO

Computação Básica



Por: Willian F. Azevedo

SUMÁRIO:

• Declaração de variáveis.....	3
• Operações matemáticas e lógicas.....	4
• Comandos básicos.....	5
• Exemplo de código (1º).....	6
• Estrutura de repetição.....	7
• Estruturas condicionais.....	9
• Array (vetores e matrizes).....	10
○ Declaração de arrays.....	10
○ Operação com arrays.....	11
• Exemplo de código(2º).....	12
• Function e subroutine.....	13
○ Function.....	13
○ Subroutine.....	14

DECLARAÇÃO DE VARIÁVEIS:

- Real x -> Variável real com 8 dígitos significativos (Ex: $x = 12.34567$)
- Real*8 x -> 16 dígitos significativos (Ex: $x = 123.456789123456$)
- Integer i -> Variável inteira (Ex: $i = 1.7$ -> $i = 1$)
- Character a -> Variável de texto (Ex: $a = \text{"Matrix"}$)
- Logical l -> Variável lógica (Ex: $l = 1 + x == 2$ -> Se $l = 2$ true, se não false)
- Complex z -> Variável complexa (Ex: Real r, i ; $z = \text{cmplx}(r, i)$)

Constante

- Parameter ($\text{pi} = 3.1415926$) -> O programa não consegue alterar " pi ", por ele ter sido definido como "parametro"

OPERAÇÕES MATEMÁTICAS E LÓGICAS:

- `_+_` -> Adição
- `_-_` -> Subtração
- `*_` -> Multiplicação
- `_/_` -> Divisão
- `**_` -> Exponenciação
- `mod(numerador, denominador)` = resto da divisão
- `L1.or.l2` -> Verdade se uma for verdade
- `L1.and.l2` -> Verdade se os dois forem verdade
- `.not. L1` -> inverte o valor lógico
- `L1.eqv.l2` -> Verdade se ambos forem iguais
- `L1.neqv.l2` -> verdade se ambos forem diferentes

COMANDOS BÁSICOS:

"Print" -> Escreve na tela, textos, resultados de operações matemáticas e valores de variáveis. Por exemplo:

- `print*, "Olá Mundo" -> "Olá mundo"` Nota: Dependendo do compilador os acentos saem bugados
- `X=1 ; print*, "O valor de x é" , x -> "O valor de x é " "1"`
- `X=1 ; print*, x + 2**2 -> "5"`

"Read" -> ""Pergunta"" para quem está rodando o código o valor de alguma variável, ou seja ele "lê" o valor dado pela pessoa. Por exemplo:

- `Print*, "Qual o valor de x" ; read*, x -> "Qual o valor de x" ____`
Nota: A parte branca é onde vai o valor escolhido, ao coloca-lo vc define o valor da variável no código, ou seja se vc escolher 3 e der `print*, x` depois do `read` o computador irá "escrever" na tela "3"

EXEMPLO DE CÓDIGO:

Observação: As variáveis devem ser declaradas no início do código, e o Fortran não diferencia letra maiúscula.

Dica: para entender o código leia como o computador leria linha por linha pensando no que o computador entende com o que está escrito.

Ex:

Implicit none

integer x, y

logical l1, l2

X=3

L1= mod((x+y)**2 *(y/x), 2) == 0 !se for verdade multiplo de 2!

L2= mod((x+y)**2 *(y/x), 3) == 0 !se for verdade multiplo de 3!

print*, "Insira valor de y "

read*, y

print*, (x+y)**2 *(y/x) ! soma elevado ao quadrado multiplicado pela divisão das variáveis!

print*, "é multiplo de 2 ou 3 ? "

print*, l1 .or. l2 !vai te responder com "T" ou "F"

end

ESTRUTURA DE REPETIÇÃO

Irá repetir a operação dentro da estrutura pela quantidade de vezes que for colocado.

Por exemplo:

X=0

- Do i=-1, 10 (irá repetir a operação 9 vezes)

X=x+2

Print*, x

enddo

-> Na Tela ("2" ; "4"; "6"; ... "18")

Obs: O "i" não precisa (mas pode ser) ser declarado como variável, ele sempre será inteiro.

O "do" com condição é similar sua única diferença é que ele repetirá a operação enquanto a condição for satisfeita .Por exemplo:

- do while (x<7)

x=x+1

print*, x

enddo

->Na tela("1" ; "2"; "3" ; ...; "7")

ESTRUTURAS CONDICIONAIS:

O "if" faz uma operação baseado na condição imposta. Por exemplo:

- If($x \geq 7$) then
 print*, x, ">= 7"
 else
 print*, x, "<= 7"
 endif

If com mais de uma condição

- If($x > 10$) then
 print*, x, "> 10"
 elseif ($x = 10$)
 print*, x, "= 10"
 else
 print*, x, "< 7"
 endif

OPERAÇÕES EM ARQUIVOS:

Primeiro é necessário abrir o arquivo, coloque o arquivo de texto na mesma pasta do arquivo ".f90".

- `Open(n, file="nome-do-arquivo")`, no qual o "n" é um número de referência do arquivo , segue exemplos:

- ```
open(1,file="Dados.txt")
do i=1,n
 read(1,*)x, y
enddo
close(2)
```

-> desta forma x é declarado como primeira coluna e y é declarado como segunda coluna, cada vez que o código executa um read de um arquivo específico ele lê a próxima linha, nesse caso o "read" funciona como se o arquivo respondesse a "pergunta".

- ```
open(2,file="Dados.txt")
write(2,*)x1, y1
write(2,*)x2, y2
write(2,*)x3, y3
Close(1)
```

-> Nesse caso, é alterado no arquivo, 3 linhas para as colunas x e y.

ARRAY (VETORES E MATRIZES):

O "array", é composto por vetores e matrizes, que podem tanto ser usados com o caráter matemático, quanto como lista.

Declaração de arrays:

- Vetores:

- Real $v(n)$ -> "n" dimensão do vetor (análogo para o "integer").
- $V=x$ -> Usado quando todas as componentes dos vetores são iguais.
- $V(n)=x_n$ -> Define o valor para cada componente "n" do vetor.
- Data $v/x_1,x_2,x_3/$

- Matrizes:

- Real $m(i,j)$ -> "i" número de linhas, e "j" número de colunas (análogo para o "integer").
- $M=x$ -> Usado quando todas as componentes da matriz são iguais.
- $m(i,j)=x_{ij}$ -> Define o valor para cada componente "i,j" da matriz.
- Data $m/x_{11},x_{12},x_{13},x_{21},x_{22},x_{23}/$ -> as componentes são colocadas coluna por coluna.

ARRAYS(VETORES E MATRIZES):

Operações com arrays:

- `Matmul(m1,m2)` -> Produto de duas matrizes ou de matriz com vetor. Ex: $m1 \times m2$.
- `Maxval(A)/ minval(A)` -> Resulta na componente de maior/menor valor de um array.
- `Dot_product(v1,v2)` -> produto interno entre dois vetores. Ex: $v1 \cdot v2$
- `Sum(A)` -> soma todas as componentes de um array.

EXEMPLO DE CÓDIGO:

Implicit none

```
integer v(4), i, y
character a(4), x
open(1,file="Gastos.txt")
|do i=1,4      !o "do" executara a operação 4x, para cada vez executada uma linha é lida!
|  read(1,*) a(i), v(i) !a cada linha lida a informação contida no arquivo é atribuída
|                                     a componente correspondente do vetor!
|enddo
close(1) !aqui o arquivo é fechado para que seja possível abri-lo de novo e voltar a ler
                                             da primeira linha!

Print*, "o gasto total foi de ", sum(v)
open(2,file="Gastos.txt")
|do i=1,4      !agora podemos reler o arquivo desde a primeira linha!
|  read(2,*) x, y
|  |if (y==maxval(v)) then !o if é usado para achar a componente "nome" correspondente
|  |                                     ao maior valor de "v"!
|  |  print*, "o item de maior gasto foi ", x
|  |endif
|enddo
end
```

FUNCTION E SUBROUTINE:

Importante no caso de precisar repetir a mesma operação diversas vezes, tal estrutura poupa tempo e linhas como por exemplo ao trabalhar com arquivos, desta forma não é mais necessário ter mais de uma linha abrindo o arquivo.

Function:

Possui esse nome pois retorna em um valor como uma função, a "function" é colocada após o fim do código principal (depois do "end") , e é referenciada no código principal com o nome que será atribuído. Por exemplo:

- Código principal

```
real m, a, force  
read*, m, a  
print*, force(m, a)  
end (código principal)
```

```
Function force(m, a)
```

```
real m, a !é necessário declarar as variáveis novamente!
```

```
force=m*a  
end
```

FUNCTION E SUBROUTINE:

Subroutine:

Similar a function entretanto não retorna um valor, está mais para um outro código inserido no principal, nele é possível fazer operações, para referenciarlo é necessário um "Call". Por exemplo:

- Código principal
real m, a, f
call force(m,a,f)
print*, f
end (código principal)

```
subroutine force(m,a,f)
real m, a, f !é necessário declarar as variáveis novamente!
print*, "insira a massa e a aceleração respectivamente"
read*, m, a
f=m*a
end
```